Experiment_9_DECISION TREE CLASSIFIER USING ID3

October 29, 2026

[6]:
```
!pip install openpyxl
```

```
Defaulting to user installation because normal site-packages is not writeable
Collecting openpyxl
  Downloading openpyxl-3.1.5-py2.py3-none-any.whl (250 kB)

250.9/250.9 KB 2.6 MB/s eta 0:00:00[31m1.8 MB/s
eta 0:00:01
Collecting et-xmlfile
  Downloading et_xmlfile-2.0.0-py3-none-any.whl (18 kB)
Installing collected packages: et-xmlfile, openpyxl
Successfully installed et-xmlfile-2.0.0 openpyxl-3.1.5
```

[10]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

file_path = "online+retail/Online Retail.xlsx"

df = pd.read_excel(file_path)

print("Original Dataset Shape:", df.shape)

df = df.dropna(subset=["CustomerID"])

df = df[~df["InvoiceNo"].astype(str).str.startswith("C")]

df = df[
    (df["Quantity"] > 0) &
    (df["UnitPrice"] > 0)
```

```
]

df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])

df["Amount"] = df["Quantity"] * df["UnitPrice"]

print("After Preprocessing:", df.shape)

reference_date = (
    df["InvoiceDate"].max()
    + pd.Timedelta(days=1)
)

customer_df = df.groupby("CustomerID").agg(

    Recency=(
        "InvoiceDate",
        lambda x: (reference_date - x.max()).days
    ),

    Frequency=(
        "InvoiceNo",
        "nunique"
    ),

    Monetary=(
        "Amount",
        "sum"
    ),

    AvgOrderValue=(
        "Amount",
        "mean"
    ),

    UniqueProducts=(
        "StockCode",
        "nunique"
    )

).reset_index()

customer_df["R_score"] = pd.qcut(
    customer_df["Recency"],
    3,
    labels=[3, 2, 1]
).astype(int)
```

```
customer_df["F_score"] = pd.qcut(
    customer_df["Frequency"].rank(method="first"),
    3,
    labels=[1, 2, 3]
).astype(int)

customer_df["M_score"] = pd.qcut(
    customer_df["Monetary"].rank(method="first"),
    3,
    labels=[1, 2, 3]
).astype(int)

customer_df["RFM_score"] = (
    customer_df["R_score"]
    + customer_df["F_score"]
    + customer_df["M_score"]
)

def assign_segment(score):

    if score >= 7:
        return "High Value"

    elif score >= 5:
        return "Medium Value"

    else:
        return "Low Value"


customer_df["Segment"] = (
    customer_df["RFM_score"]
    .apply(assign_segment)
)


print("\nCustomer Segments:")
print(customer_df["Segment"].value_counts())

features = [
    "Recency",
    "Frequency",
    "AvgOrderValue",
    "UniqueProducts"
]
```

```
X = customer_df[features]

y = customer_df["Segment"]

X_train, X_test, y_train, y_test = train_test_split(

    X,
    y,

    test_size=0.20,

    random_state=42,

    stratify=y
)


print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))

model = DecisionTreeClassifier(

    criterion="entropy",

    max_depth=4,

    min_samples_split=10,

    random_state=42
)


# Train model
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(
    y_test,
    y_pred
)

print("\n======================================")
print("MODEL PERFORMANCE")
print("======================================")

print("Accuracy:", round(accuracy, 4))
```

```
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred
    )
)

print("\nConfusion Matrix:")
print(
    confusion_matrix(
        y_test,
        y_pred
    )
)

importance_df = pd.DataFrame({

    "Feature": features,

    "Importance": model.feature_importances_

}).sort_values(
    "Importance",
    ascending=False)

print("\n=====================================")
print("FEATURE IMPORTANCE")
print("=====================================")

print(
    importance_df.to_string(
        index=False
    )
)

plt.figure(figsize=(8, 5))

plt.bar(
    importance_df["Feature"],
    importance_df["Importance"]
)

plt.xlabel("Features")

plt.ylabel("Importance")
```

```
plt.title(
    "Decision Tree - Feature Importance"
)

plt.xticks(rotation=30)

plt.tight_layout()

plt.show()

plt.figure(figsize=(20, 10))

plot_tree(

    model,

    feature_names=features,

    class_names=model.classes_,

    filled=True,

    rounded=True,

    fontsize=9

)

plt.title(
    "ID3 Decision Tree for Customer Segmentation"
)

plt.show()

print("\n=====================================")
print("EXPERIMENT SUMMARY")
print("=====================================")

print("Number of Customers:",
      len(customer_df))

print("Number of Features:",
      len(features))

print("Target Variable: Segment")
```

```python
print("Algorithm: ID3 Decision Tree")

print("Criterion: Entropy")

print("Accuracy:",
      round(accuracy, 4))

print(
    "\nMost Important Feature:",
    importance_df.iloc[0]["Feature"]
)
```

```
Original Dataset Shape: (541909, 8)
After Preprocessing: (397884, 9)

Customer Segments:
Segment
High Value      1810
Medium Value    1286
Low Value       1242
Name: count, dtype: int64

Training Samples: 3470
Testing Samples: 868

====================================
MODEL PERFORMANCE
====================================
Accuracy: 0.8744

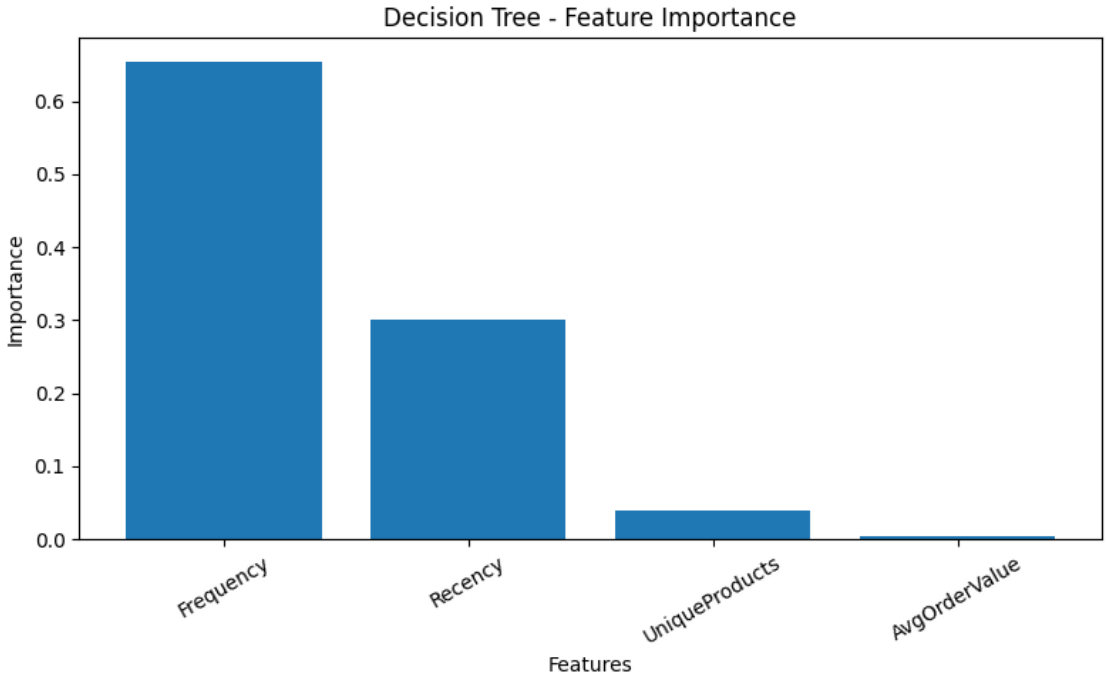
Classification Report:
              precision    recall  f1-score   support

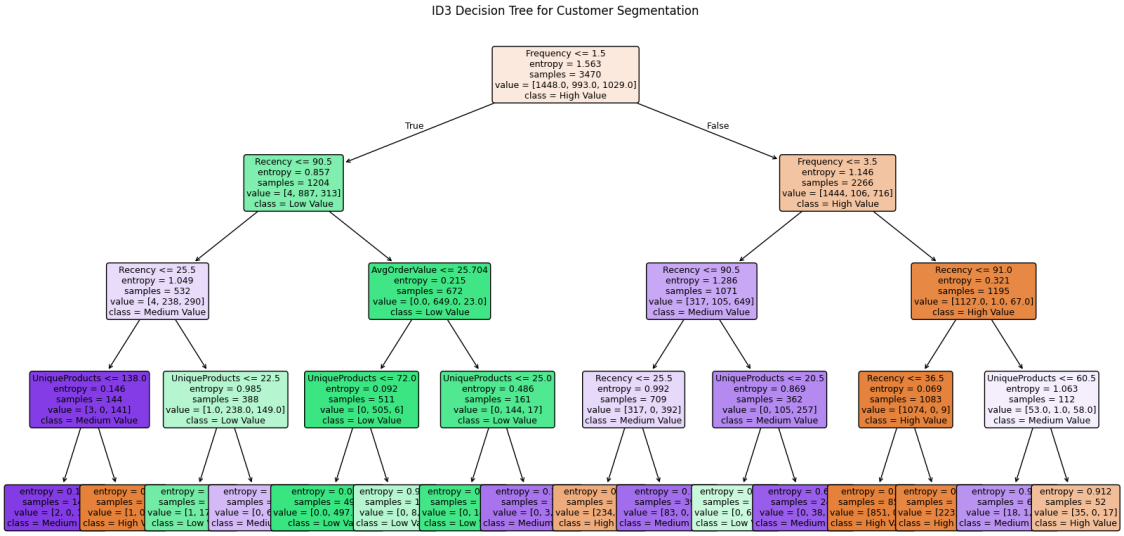
  High Value       0.92      0.94      0.93       362
   Low Value       0.85      0.91      0.88       249
Medium Value       0.82      0.74      0.78       257

    accuracy                           0.87       868
   macro avg       0.87      0.87      0.86       868
weighted avg       0.87      0.87      0.87       868


Confusion Matrix:
[[342   0  20]
 [  0 227  22]
 [ 28  39 190]]
```

```
======================================
FEATURE IMPORTANCE
======================================
       Feature  Importance
     Frequency    0.653876
       Recency    0.301166
UniqueProducts    0.040184
 AvgOrderValue    0.004773
```





```
======================================
EXPERIMENT SUMMARY
======================================
Number of Customers: 4338
Number of Features: 4
Target Variable: Segment
Algorithm: ID3 Decision Tree
Criterion: Entropy
Accuracy: 0.8744

Most Important Feature: Frequency
```

[ ]:

[ ]: